

R1-A3-S7 Redes Neuronales III

Edward Alexander Moreno Matallana

Universidad de Cundinamarca UCUNDINAMARCA

Especialización en Inteligencia Artificial

Programa

2026

Resumen

Este informe presenta el desarrollo de una práctica de clasificación de dígitos manuscritos con MNIST mediante TensorFlow y Keras. Se documentan la partición estratificada de los datos, la normalización de píxeles, una red densa de referencia, la comparación de optimizadores y tasas de aprendizaje, y un experimento de regularización con L2, Dropout y Early Stopping.

Palabras clave: MNIST, redes neuronales, optimización, regularización, evaluación, generalización.

Abstract

This report details the development of a handwritten digit classification task using the MNIST dataset with TensorFlow and Keras. It documents stratified data partitioning, pixel normalization, a baseline dense network, a comparison of optimizers and learning rates, and a regularization experiment involving L2, Dropout, and Early Stopping.

Keywords: MNIST, neural networks, optimization, regularization, evaluation, generalization.

Tabla de Contenido

Introducción	4
Objetivos	5
Objetivo General.....	5
MARCO CONCEPTUAL	6
Hiperparámetros	6
Optimización	6
Regularización	6
DATOS Y PROCESAMIENTO.....	7
MODELO BASELINE	8
OPTIMIZACIÓN E HIPERPARÁMETROS.....	10
REGULARIZACIÓN	13
DIAGNOSTICO BIAS	14
EVALUACIÓN FINAL.....	15
MATRIZ DE CONFUSIÓN Y ANALISIS DE ERROR.....	17
IMPLEMENTACIÓN CON PyTORCH	19
CONCLUSIONES	21
ANEXO.....	22

Introducción

La construcción de una red neuronal requiere más que elegir una arquitectura. Las decisiones sobre los datos, el optimizador, la tasa de aprendizaje y la regularización afectan tanto el proceso de ajuste como la capacidad de generalizar. Por ello, esta práctica compara configuraciones de entrenamiento y reserva un conjunto de prueba para estimar el desempeño final del clasificador.

El problema abordado es la clasificación de imágenes en escala de grises de dígitos manuscritos de 0 a 9. El cuaderno utiliza una red neuronal densa con imágenes de 28×28 píxeles aplanadas en vectores, entrena modelos con Keras y calcula métricas globales y por clase. También define un modelo equivalente en PyTorch, sin mostrar su entrenamiento ni evaluación.

Objetivos

Objetivo General

Implementar y analizar un flujo de clasificación de MNIST que aplique preprocesamiento, optimización, regularización y evaluación de modelos de redes neuronales.

MARCO CONCEPTUAL

Hiperparámetros

Son decisiones fijadas antes o durante el entrenamiento, por ejemplo learning rate, tamaño del lote, número de unidades, número de épocas, optimizador y fuerza de regularización. No se estiman directamente por backpropagation; se comparan con el conjunto dev.

Optimización

Actualiza los pesos para minimizar la función de pérdida. SGD con momentum incorpora inercia, RMSprop adapta el paso por parámetro y Adam combina momentos de primer y segundo orden. Un learning rate demasiado bajo progresa lentamente; uno alto puede producir oscilaciones o perder precisión.

Regularización

Limita la complejidad efectiva: L2 penaliza pesos grandes, Dropout desactiva activaciones de forma aleatoria durante entrenamiento y Early Stopping interrumpe cuando la pérdida de dev deja de mejorar. Bias alto sugiere underfitting; variance alta aparece cuando train

supera mucho a dev. Accuracy resume aciertos globales; Precision, Recall y F1 hacen visible el comportamiento por clase.

DATOS Y PROCESAMIENTO

MNIST contiene imágenes en escala de grises de 28 x 28 píxeles y etiquetas enteras de 0 a 9. Los 60 000 ejemplos originales de entrenamiento se dividieron de forma estratificada en 48 000 para train y 12 000 para dev (20 %). Los 10 000 ejemplos oficiales de test se conservaron para la evaluación final. Cada imagen se convirtió a float32 y se normalizó de [0, 255] a [0, 1].

Conjunto	Forma de X	Forma de y	Uso
Train	48000 x 28 x 28	48000	Ajuste de pesos
Dev	12000 x 28 x 28	12000	Selección y control
Test	10000 x 28 x 28	10000	Evaluación final

Tabla. La partición evita que el conjunto de prueba influya en la selección de hiperparámetros. La estratificación preserva proporciones similares de dígitos en train y dev.

Dígito	Train	Dev	Test
0	4 738 (9.87 %)	1 185 (9.88 %)	980 (9.80 %)
1	5 394 (11.24 %)	1 348 (11.23 %)	1 135 (11.35 %)
2	4 766 (9.93 %)	1 192 (9.93 %)	1032 (10.32 %)
3	4 905 (10.22 %)	1 226 (10.22 %)	1 010 (10.10 %)
4	4 674 (9.74 %)	1 168 (9.73 %)	982 (9.82 %)

5	4 337 (9.04 %)	1 084 (9.03 %)	892 (8.92 %)
6	4 734 (9.86 %)	1 184 (9.87 %)	958 (9.58 %)
7	5 012 (10.44 %)	1 253 (10.44 %)	1 028 (10.28 %)
8	4 681 (9.75 %)	1 170 (9.75 %)	974 (9.74 %)
9	4 759 (9.91 %)	1 190 (9.92 %)	1 009 (10.09 %)

Tabla. La distribución es aproximadamente uniforme y train/dev conservan prácticamente el mismo porcentaje por clase. Por tanto, una diferencia de métricas no se atribuye a un desbalance introducido por la partición.

MODELO BASELINE

El baseline es una red densa 784 - 128 - 10: Flatten transforma la imagen en 784 características, una capa oculta Densecon ReLU aprende representaciones y Softmax entrega probabilidades de las diez clases. Con 101 770 parámetros entrenables, Adam (learning rate 0.001), batch size 64 y cinco épocas, sirve como referencia común para los experimentos.

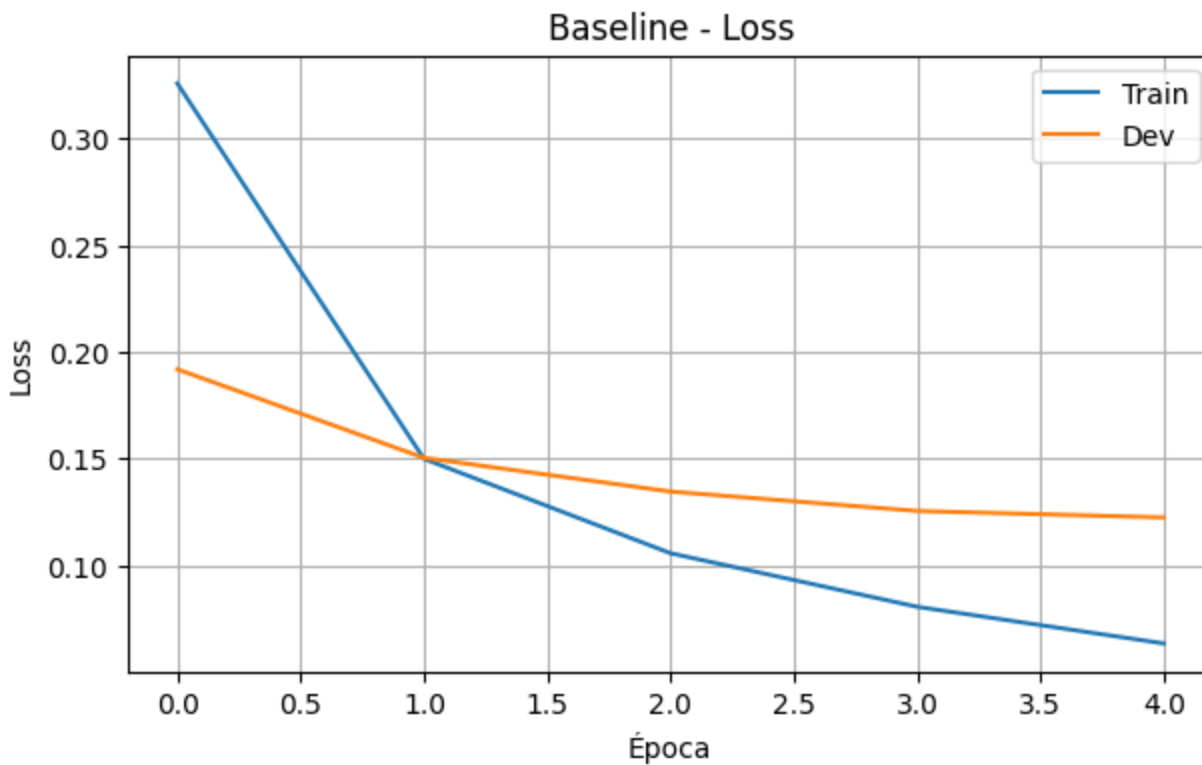


Figura. Curva de pérdida del baseline. La pérdida de train y dev disminuye durante las cinco épocas; la curva de dev permite comprobar que la mejora se conserva fuera de los lotes usados para ajustar pesos.

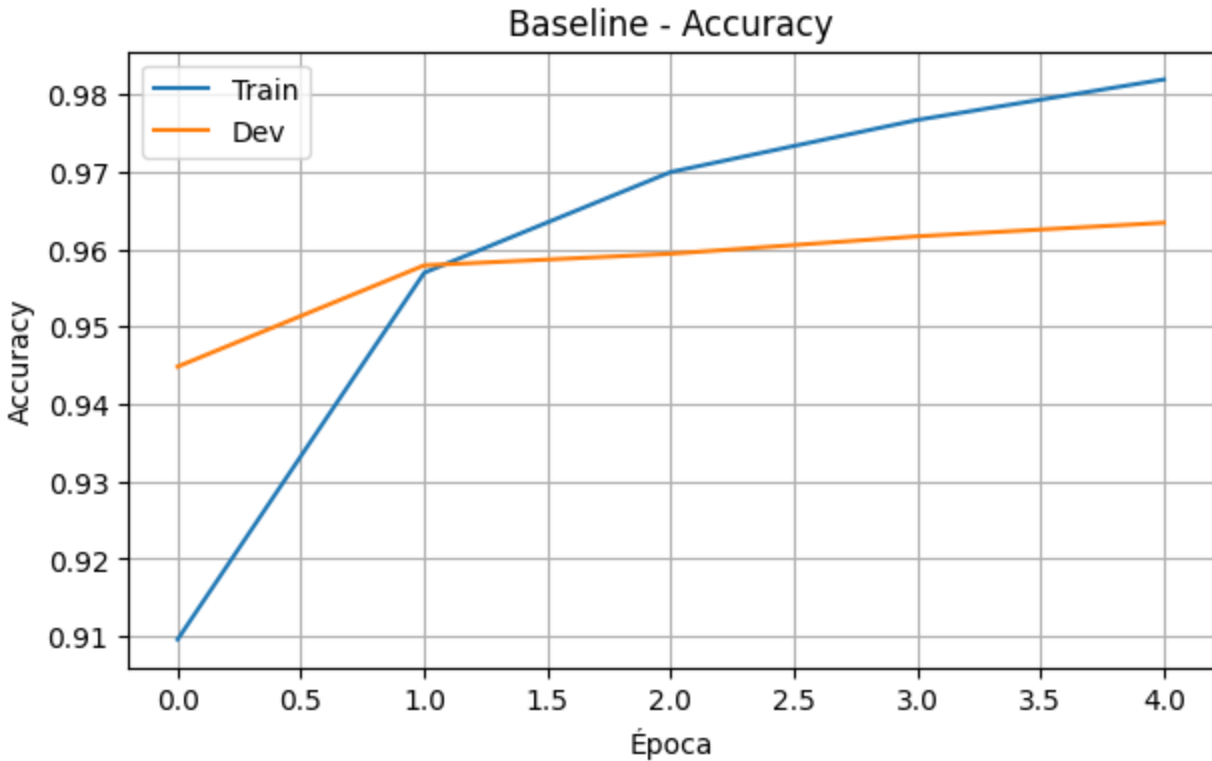


Figura. Curva de accuracy del baseline. Su lectura conjunta con la pérdida permite detectar si el aumento en train deja de transferirse al conjunto dev.

OPTIMIZACIÓN E HIPERPARÁMETROS

Para aislar el efecto del optimizador se mantuvieron la arquitectura, los datos, el batch size (64) y cinco épocas. RMSprop registró la menor dev loss (0.109351) y el mayor dev accuracy (0.968250) entre los tres métodos. Adam logró mayor accuracy de train, pero RMSprop generalizó ligeramente mejor en esta ejecución.

optimizador	train_accuracy	dev_accuracy	dev_loss	Interpretación
SGD + momentum	0.966271	0.958250	0.141563	Convergencia correcta, pero menor desempeño de dev.
RMSprop	0.979792	0.968250	0.109351	Menor pérdida y mejor accuracy de dev
Adam	0.982083	0.964667	0.117832	Excelente ajuste de train; dev ligeramente inferior a RMSprop.

Tabla. Comparación bajo condiciones iguales. La decisión se apoya en dev loss y dev accuracy, no solo en la métrica de entrenamiento.

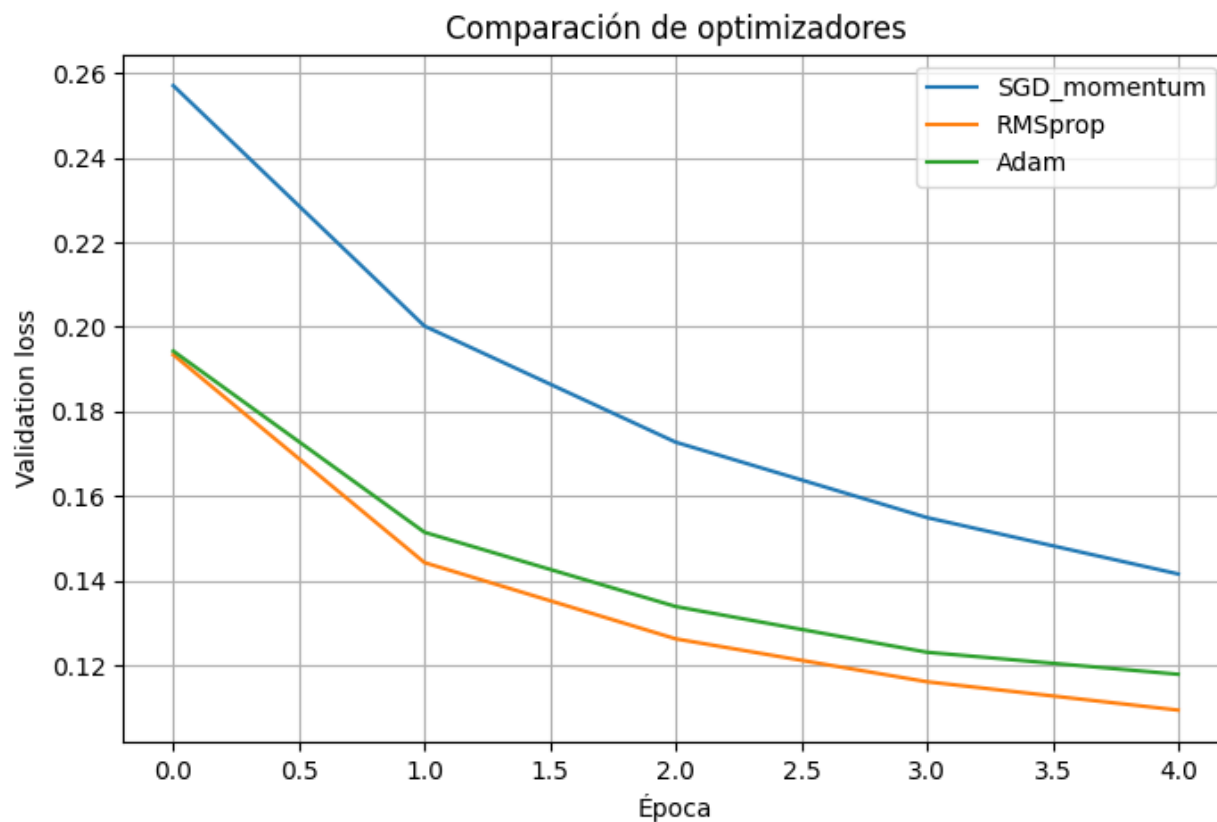


Figura. Pérdida de validación por época. RMSprop termina en la región más baja de la gráfica, evidencia visual coherente con la tabla de resultados.

Con Adam se evaluaron tres learning rates durante tres épocas. El paso 0.0001 produjo dev loss 0.281903 y dev accuracy 0.922833: el aprendizaje fue demasiado lento para el presupuesto de épocas. El valor intermedio 0.001 obtuvo la mejor dev loss (0.131901) y dev accuracy (0.960167). Con 0.01 la dev loss subió a 0.154741, señal de actualizaciones menos estables.

learning_rate	train_accuracy	dev_accuracy	dev_loss	Decisión
---------------	----------------	--------------	----------	----------

0.0001	0.918500	0.922833	0.281903	Paso insuficiente en 3 épocas
0.0010	0.969500	0.960167	0.131901	Mejor equilibrio
0.0100	0.968521	0.959250	0.154741	Paso más agresivo; mayor pérdida

La tabla revela el tradeoff: una tasa demasiado pequeña aprende con lentitud y una grande no ofrece la menor pérdida de desarrollo. Por ello se conserva 0.001 como valor de referencia.

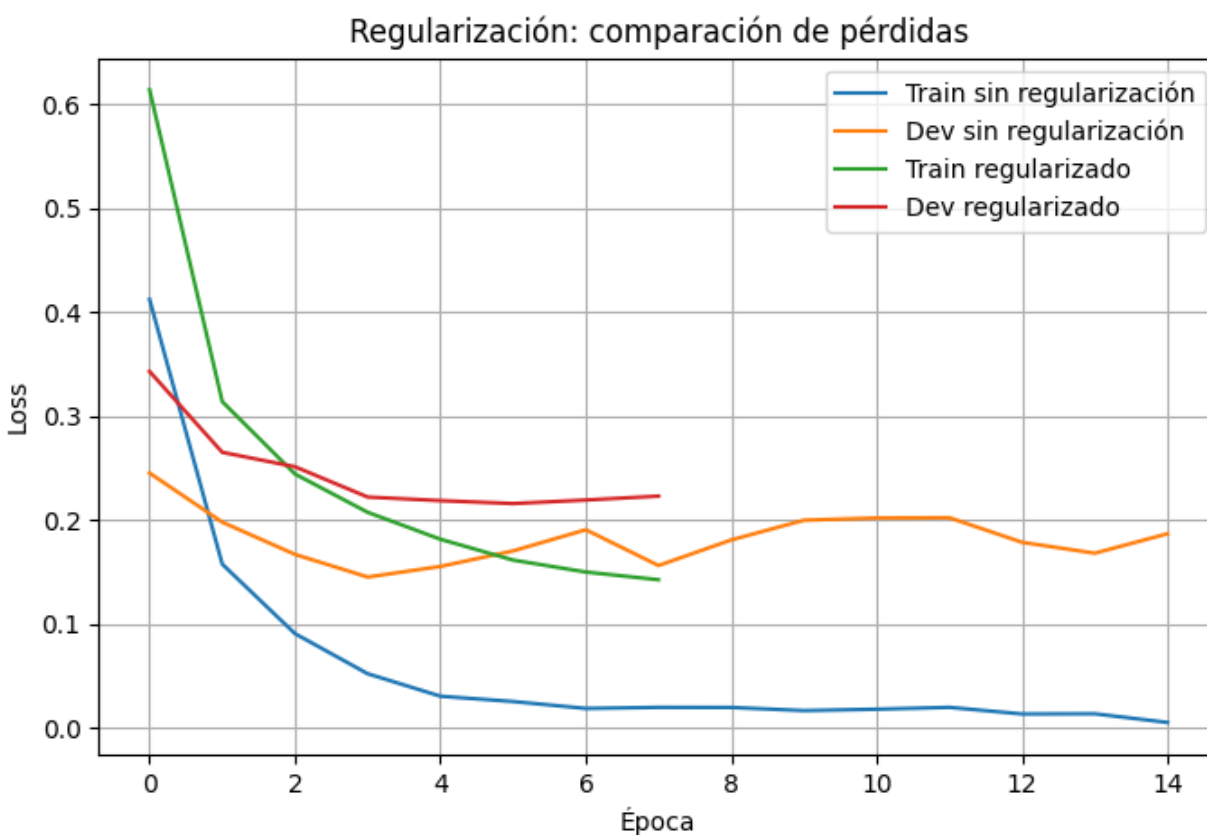
REGULARIZACIÓN

El experimento de regularización usa una muestra estratificada de 12 000 imágenes de train para hacer visible el sobreajuste en una red de mayor capacidad (512 y 256 unidades). La versión sin regularización se entrenó 15 épocas. La versión regularizada mantiene la arquitectura, agrega $L2 = 1e-4$ en ambas capas densas, Dropout = 0.30 después de cada capa y Early Stopping sobre val_loss con patience = 2 y restauración de los mejores pesos.

Modelo	Dev loss	Dev accuracy	Épocas	Lectura
Sin regularización	0.186545	0.965333	15	Ajuste de dev algo mayor, con brecha de generalización amplia.

L2 + Dropout + Early Stopping	0.215628	0.962750	8	Menor brecha train-dev y detención automática; costo leve en dev.
----------------------------------	----------	----------	---	---

Tabla. Early Stopping detuvo el entrenamiento en la época 8 porque val_loss no mejoró durante dos épocas consecutivas. El resultado enseña que regularizar busca controlar la generalización, no maximizar necesariamente la accuracy de una ejecución de dev.



Curvas de pérdida con y sin regularización. La separación entre train y dev de la red no regularizada evidencia que el ajuste del conjunto de entrenamiento avanza más que su capacidad de generalizar.

DIAGNOSTICO BIAS

La brecha train-dev se calcula como $\text{accuracy_train} - \text{accuracy_dev}$. Debe leerse junto con la capacidad y el número de épocas: el modelo pequeño solo se entrenó dos épocas, por lo que su resultado sugiere underfitting o entrenamiento insuficiente, no una conclusión causal aislada.

Modelo	Train accuracy	Dev accuracy	Brecha	Diagnóstico
Pequeño: 16 unidades	0.921292	0.924667	-0.003375	Bajo desempeño en ambos conjuntos: bias alto o entrenamiento insuficiente.
Grande sin regularización	0.998750	0.965333	0.033417	Variance alta: ajuste casi perfecto de train y caída en dev.
Grande regularizado	0.981167	0.962000	0.019167	Menor variance; conserva desempeño competitivo de dev.

La regularización reduce la brecha de 3.34 a 1.92 puntos porcentuales. Esta evidencia apoya elegir un modelo con mejor control de variance cuando la prioridad es generalizar.

EVALUACIÓN FINAL

El modelo final de Keras conserva la arquitectura regularizada y se entrena con los 48 000 ejemplos de train, usando dev para Early Stopping. Solo después se evalúa una vez sobre test. Esta separación protege la estimación final contra decisiones repetidas guiadas por el conjunto de prueba.

Métrica	Resultado	Qué permite concluir
Loss	0.158200	Magnitud promedio del error probabilístico.
Accuracy	0.978300	97.83 % de los 10 000 dígitos fue clasificado correctamente.
Precision macro	0.978169	Promedio equilibrado de exactitud entre clases predichas.
Recall macro	0.978063	Promedio equilibrado de cobertura de las clases reales.
F1 macro	0.978108	Balance entre Precision y Recall para las diez clases.

Las métricas macro son muy próximas a Accuracy; junto con la distribución de clases, esto indica que el rendimiento no depende de una clase dominante.

El reporte por clase confirma resultados altos para todos los dígitos. La clase 6 tiene el F1 más bajo (0.9740) en esta ejecución, aun así cercano al promedio macro. Conviene revisar errores concretos, ya que un único promedio puede ocultar pares de dígitos visualmente similares.

Dígito	Precisión	Recall	F1	Soporte
0	0.9887	0.9857	0.9872	980
1	0.9860	0.9947	0.9904	1135

2	0.9824	0.9758	0.9791	1032
3	0.9763	0.9792	0.9778	1010
4	0.9806	0.9766	0.9786	982
5	0.9775	0.9753	0.9764	892
6	0.9710	0.9770	0.9740	958
7	0.9794	0.9708	0.9751	1028
8	0.9712	0.9702	0.9707	974
9	0.9685	0.9752	0.9719	1009

Reporte de clasificación sobre los 10 000 ejemplos de test. El soporte es el número de imágenes reales de cada clase y permite interpretar las métricas con su tamaño de muestra.

MATRIZ DE CONFUSIÓN Y ANALISIS DE ERROR

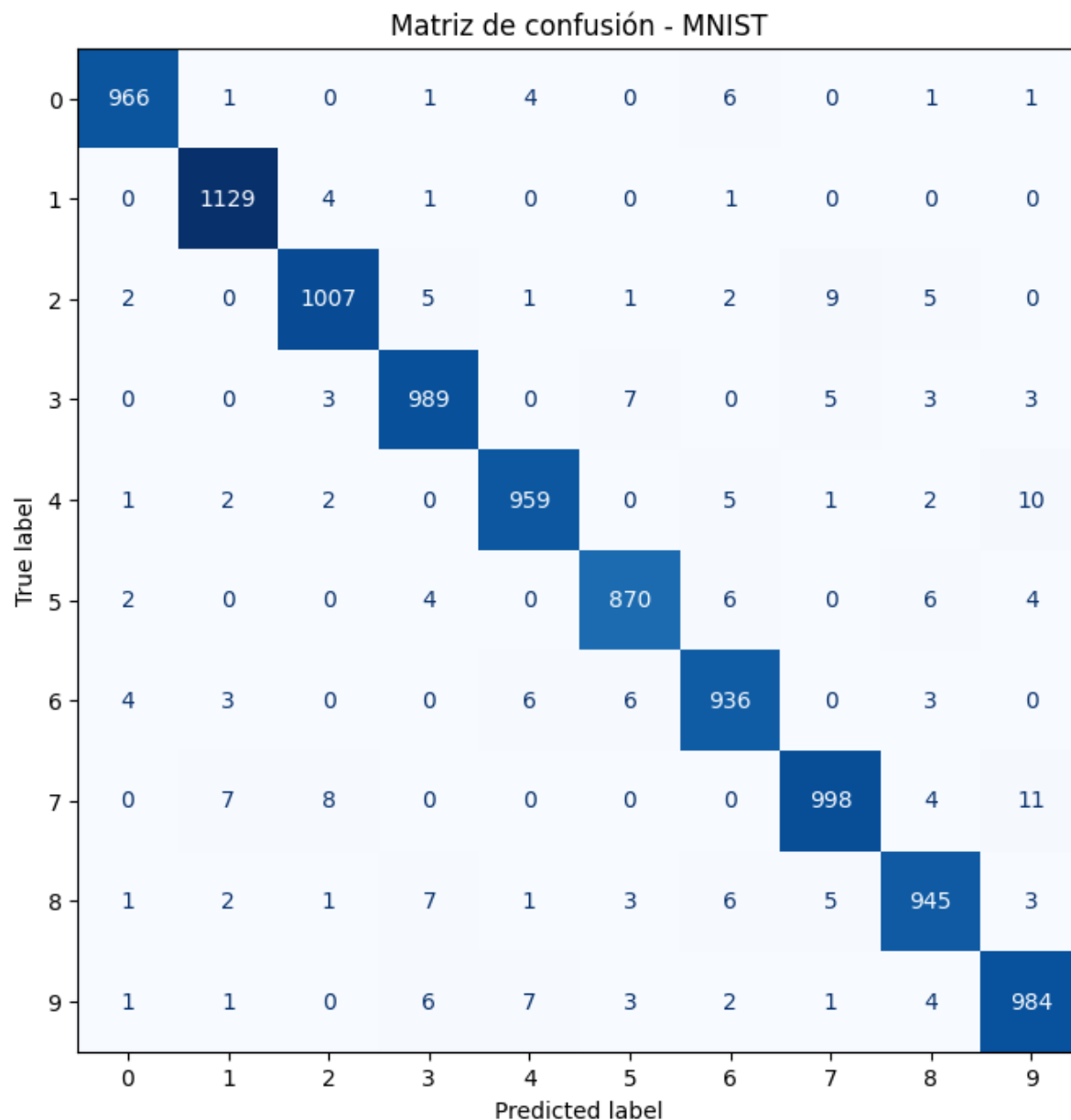
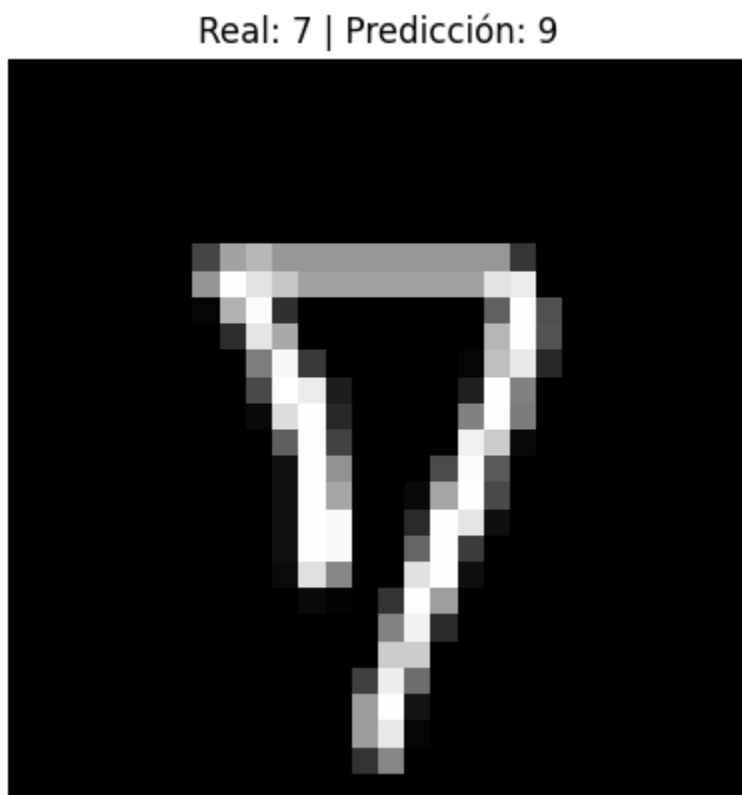


Figura. Matriz de confusión del modelo final. La diagonal concentra los aciertos; las celdas fuera de la diagonal son los errores y orientan la inspección de clases similares.

La confusión más frecuente fue 7 clasificado como 9, con 11 casos. Este dato convierte la matriz en una acción concreta: se revisa un ejemplo real de 7 para determinar si el trazo superior

y el descendente se parecen visualmente a un 9. No es una afirmación de que todos los siete fallan, sino el par de error más repetido de la corrida.



Ejemplo de error asociado a la confusión 7 -> 9. La figura permite inspeccionar la ambigüedad del trazo y plantear mejoras de datos, por ejemplo, aumentación moderada o una arquitectura convolucional.

IMPLEMENTACIÓN CON PyTORCH

Como contraste de herramientas, se implementó una red densa equivalente en PyTorch: Flatten, Linear 784 -> 256, ReLU, Dropout 0.30 y Linear 256 -> 10. Se utilizó

CrossEntropyLoss, Adam con learning rate 0.001 y weight_decay 1e-4 (regularización L2). El entrenamiento se realizó en CPU durante cinco épocas, y el test se consultó solo al terminar.

Framework	Modelo evaluado	Resultado final	Lectura
TensorFlow + Keras	Regularizado 512 - 256 con L2 y Dropout	Test accuracy: 0.9783	API de alto nivel; facilita experimentos y callbacks
PyTorch	256 unidades con Dropout y weight decay	Test accuracy: 0.9738	Ciclo de entrenamiento explícito; control fino de tensores y gradientes.

Los resultados provienen de arquitecturas y ejecuciones distintas, por lo que no constituyen una prueba estadística de superioridad entre frameworks. Ambos implementan correctamente el flujo train-dev-test y superan 97% de accuracy en test.

CONCLUSIONES

La separación estratificada 48 000/12 000/10 000 y la normalización a $[0, 1]$ establecieron una base válida para comparar modelos sin contaminar test.

En la comparación controlada, RMSprop entregó la menor dev loss (0.109351). Para Adam, 0.001 fue mejor que 0.0001 y 0.01 bajo el presupuesto fijado, lo que evidencia que el learning rate requiere ajuste empírico.

La red grande sin regularización alcanzó más accuracy de dev en ese experimento, pero mostró mayor brecha train-dev. L2, Dropout y Early Stopping redujeron la brecha de 3.34 a 1.92 puntos porcentuales, con una leve disminución de accuracy de dev. La decisión depende de priorizar generalización estable sobre el máximo de una sola corrida.

El modelo final regularizado obtuvo 0.9783 de accuracy y F1 macro 0.9781 en test. El error más frecuente fue 7 -> 9 (11 casos); una mejora futura razonable es evaluar una CNN y aumento de datos, comparándolos con el mismo protocolo y más de una semilla.

ANEXO

Los notebooks ejecutados y el código fuente se encuentran en el repositorio público:

<https://github.com/edwardmoreno18/DeepLearning>. Los experimentos documentados corresponden a Baseline en Keras, Optimización de hiperparámetros, Regularización, Métricas de evaluación y PyTorch.